

Delpat Engineering: Onboarding & Operational Guidelines

1. Engineering Philosophy

Welcome to the Delpat engineering team. This document serves as the operational blueprint for how we build software. Our goal during your onboarding is to help you transition into our specific workflow, toolset, and coding standards.

We value three core principles above all else:

- **Communication:** In a distributed team, we cannot see you working. Communication is your presence. It is better to over-communicate your status than to remain silent.
- **Punctuality:** We respect your time and expect the same in return. Being online during core hours and attending scheduled syncs is mandatory.
- **Intellectual Honesty:** We prioritize understanding over speed. It is acceptable to not know something; it is unacceptable to hide that fact.

2. Artificial Intelligence Policy

We encourage the use of AI tools (ChatGPT, Claude, Copilot) to improve efficiency. However, usage must be deliberate and responsible.

The Rule of Understanding You are responsible for every line of code you submit. You must understand the logic, syntax, and potential side effects of any code, regardless of its origin. If asked, "How does this function work?", answering "AI wrote it" is not acceptable.

Guidelines:

- **Encouraged:** Using AI to generate boilerplate, write unit tests, debug complex error logs, or explain documentation.
- **Prohibited:** Copy-pasting code blocks without reviewing them, or asking AI to design high-level architecture without your own engineering input.

3. Tool Stack & Usage

We maintain strict separation of concerns for our tools to keep information organized.

Discord (Communication)

- **Usage:** Used for real-time communication, daily stand-ups, and quick queries.
- **Expectation:** You should be reachable on Discord during core working hours.

Notion (Documentation)

- **Usage:** Stores project specifications, technical documentation, and the knowledge base.
- **Expectation:** Always check Notion for existing documentation before asking a question.

GitHub (Code Management)

- **Usage:** The single source of truth for our codebase and task management.
- **Expectation:** All work must be tracked via Issues and Pull Requests. No code exists outside of GitHub.

Voice Channels (Collaboration)

- **Usage:** Utilized for daily syncs, pair programming, and complex debugging sessions.
- **Expectation:** Use voice channels when text communication becomes inefficient.

Delpat OS (Reporting)

- **Usage:** Put daily reports and meeting notes on OS everytime.
- **Expectation:** After every meet and day completion you are supposed to update.

4. Operational Schedule

- **Core Hours Time:** Monday to Saturday, 9Am - 1Pm , 2Pm - 6Pm , 7Pm - 11 Pm (Any two slots)
- During this window, the team works synchronously. You must be online and responsive. Outside these hours, you may work asynchronously to meet your targets.

Daily Reporting To ensure alignment without micromanagement, we follow a simple daily routine:

1. **Morning Sync:** A brief discussion at the start of core hours to align on the day's objectives and identify any blockers.
2. **End of Day (EOD) Report:** Before signing off, update to OS .

5. Development Workflow

We follow a standard lifecycle for every task to ensure quality and consistency.

Step 1: Requirement Analysis Before writing code, ensure you understand the requirements. Review the ticket, wireframes, and technical specs. Clarify any ambiguities immediately.

Step 2: Task Assignment Move your designated ticket on the Project Board to "In Progress" to signal you have started work.

Step 3: Development Create a new branch for your task (refer to Git Guidelines). Write your code and verify it locally.

Step 4: Code Review Submit a Pull Request (PR) on GitHub.

Step 5: Merge & Deploy Code is merged into the main codebase only after it has been reviewed and approved by a lead.

6. Troubleshooting Protocol

Software engineering involves constant problem-solving. When you get stuck, follow this protocol to resolve issues efficiently.

The 30-Minute Timebox

1. Spend 30 minutes attempting to solve the problem yourself. Read error logs, check documentation, and research the issue.
2. If the issue remains unresolved after 30 minutes, you are required to ask for help in the `#dev-help` channel.

Request Format

- **Goal:** What you are trying to achieve.
- **Blocker:** The specific error or issue preventing progress (include logs).
- **Attempt:** What steps you have already taken to fix it.

7. Architecture & Coding Standards

General Principles

- **Clarity:** Write code that is easy for humans to read. Avoid overly complex logic where simple logic will suffice.
- **Modularity:** Functions should have a single responsibility. Separate business logic from user interface code.
- **DRY (Don't Repeat Yourself):** Refactor repeated logic into reusable functions.
- **Comments:** Comments should explain the *why* (context), not the *what* (syntax).

Technical Stack

- **Frontend:** Component-based architecture.
- **Backend:** Layered architecture (Routes → Controllers → Services).
- **AI/ML:** Pipeline-based architecture (Data Processing → Model Training → Inference).

8. Git Guidelines

We maintain a clean version history to ensure our codebase is manageable.

Branch Naming Structure: `type/ticket-id-description`

- `feat`: New features
- `fix`: Bug fixes
- `chore`: Maintenance or cleanup
- `exp`: Experiments or research trials
- *Example*: `feat/T-123-login-page`

Commit Messages Use "Conventional Commits" format: `type(scope): subject`

- *Example*: `feat(auth): implement password hashing`
- *Example*: `fix(ui): correct mobile alignment on dashboard`

9. Security & Secrets

- **No Hardcoded Secrets**: Never commit API keys, passwords, or credentials to GitHub.
- **Environment Variables**: Use `.env` files for local secrets. Ask your lead for the `.env.example` file if one is not provided.

10. Onboarding Roadmap

Your lead will assign you a specific starter project. This project is designed to help you set up your environment and get comfortable with our workflow.

- **Phase 1: Setup & Configuration**: Setting up your local development environment, gaining access to necessary tools, and completing your first repository contribution.
- **Phase 2: Core Development**: executing the main requirements of your assigned project.
- **Phase 3: Review & Refine**: Cleaning up code, writing documentation, and walking through your work with a lead.

11. Day 1 Immediate Checklist

Please complete the following items upon joining:

1. **Account Setup:** Accept invites to Discord, Notion, and GitHub.
2. **Introduction:** Post a brief introduction in the [#introductions](#) channel on Discord.
3. **Documentation:** Read this handbook thoroughly and review the Project Brief provided by your lead.
4. **Local Environment:** Clone the repository and follow the installation instructions in the [README.md](#).
5. **First Contribution:** Create a new branch, add your name to the [CONTRIBUTORS.md](#) file, commit it using the guidelines above, and open your first Pull Request.

Welcome to Delpat. Let's build.